Lecture 5.1 – Digital Control Implementation

Week 5: Sampling, Timing, and Discrete-Time Control

Learning Objectives

By the end of this lecture, you should be able to:

- explain what a sample time is;
- describe how Arduino implements feedback control digitally;
- calculate sample frequency from sample time;
- explain why sample time affects controller behavior;
- write the discrete PI update equations;
- describe practical timing and serial-data issues.

1. From Continuous Control to Digital Control

So far, we wrote controllers in continuous-time form.
For PI control,

$$u(t) = K_p e(t) + K_i \int e(t)\, dt.$$

But the Arduino does not calculate continuously. It calculates at separate time instants:

$$t = 0,\ T_s,\ 2T_s,\ 3T_s, \ldots$$

where T_s is the sample time.

> A digital controller measures, calculates, and updates the motor command once every sample period.

2. Sample Time and Sample Frequency

The sample time is

$$T_s.$$

The sample frequency is

$$\boxed{f_s = \frac{1}{T_s}.}$$

Examples:

Sample time T_s	Sample frequency f_s	Meaning
0.02 s	50 Hz	fast update
0.05 s	20 Hz	moderate update
0.10 s	10 Hz	simple lab update
0.20 s	5 Hz	slow update

For this motor project, $T_s = 0.10$ s is a reasonable starting point because the motor response is not extremely fast and it makes data logging easier.

3. What Happens in One Control Cycle?

At each sample time, the Arduino performs these steps:

1. count encoder pulses since the last update;
2. convert counts to RPM;
3. optionally filter the RPM estimate;
4. calculate error:
$$e[k] = r[k] - y[k];$$
5. update the integral:
$$I[k] = I[k-1] + e[k]T_s;$$
6. compute:
$$u[k] = K_p e[k] + K_i I[k];$$
7. clamp PWM to 0–255;
8. send the new PWM to the motor driver.

4. Discrete-Time PI Equations

The discrete PI controller is

$$\boxed{e[k] = r[k] - y[k]}$$

$$\boxed{I[k] = I[k-1] + e[k]T_s}$$

$$\boxed{u[k] = K_p e[k] + K_i I[k].}$$

The index k means "current sample."

The index $k-1$ means "previous sample."

The controller remembers the previous integral value and updates it using the current error.

5. Why Sample Time Matters

If the sample time is too large:

- the controller reacts slowly;
- RPM estimates may look jumpy;
- the motor may overshoot before the controller reacts;
- disturbance recovery may be delayed.

If the sample time is too small:

- there may be very few encoder counts per sample;
- RPM estimates may become noisy;
- serial printing may interfere with timing;
- the Arduino may spend too much time logging data.

A faster sample time is not automatically better. It must still produce reliable measurements and stable controller behavior.

6. Encoder Counts and Measurement Resolution

The RPM estimate is based on encoder counts during the sample period.
If T_s is short, the number of counts in one sample may be small.
For example, suppose the motor speed gives about 500 counts per second.
With

$$T_s = 0.10 \text{ s},$$

the sample gets about

$$500(0.10) = 50$$

counts.
With

$$T_s = 0.02 \text{ s},$$

the sample gets about

$$500(0.02) = 10$$

counts.
Fewer counts can make the RPM estimate more jumpy.

7. Timing in Arduino Code

A good digital controller should update at regular intervals.
The typical approach is:

```
if (now - lastSample >= SAMPLE_MS) {
    // measure speed
    // update controller
    // apply PWM
    // print data
    lastSample = now;
}
```

This is better than using long blocking delays because the controller timing remains more predictable.

8. Serial Printing Can Affect Timing

Serial output is useful for data logging, but it takes time.
If too much text is printed each sample, the controller may run slower than expected.
For the labs, keep serial output compact:

```
time, reference, rpm, rpm_filtered, error, pwm, integral
```

Use simple CSV-style data so Python can record it without slowing the controller too much.

9. Practical Sample-Time Choice

For our class motor project:

$$T_s = 0.10 \text{ s}$$

is a good default.
It gives:

$$f_s = 10 \text{ Hz}.$$

This means the controller updates 10 times per second.
That is usually enough for this small motor-speed project and keeps the data easy to analyze.

10. Check Your Understanding

Answer briefly.

1. If $T_s = 0.05$ s, what is the sample frequency?
2. If the encoder produces 600 counts/s, about how many counts occur in 0.10 s?
3. Why can a very small T_s make RPM noisy?
4. Why is serial output useful in this course?
5. Why should serial output be kept compact?

Summary

- Arduino implements feedback control digitally.
- The controller updates once every sample time T_s.
- Sample frequency is $f_s = 1/T_s$.
- Discrete PI uses
$$I[k] = I[k-1] + e[k]T_s.$$
- Sample time affects response speed, noise, and measurement quality.
- CSV-style serial output allows Python to record controller data.